[Nam Le] Boot Process (ARM), U-Boot, Linux Kernel, Root Filesystem & Buildroot

1. The boot process of an Embedded platform

1.1 What does a bootloader (U-boot) do?

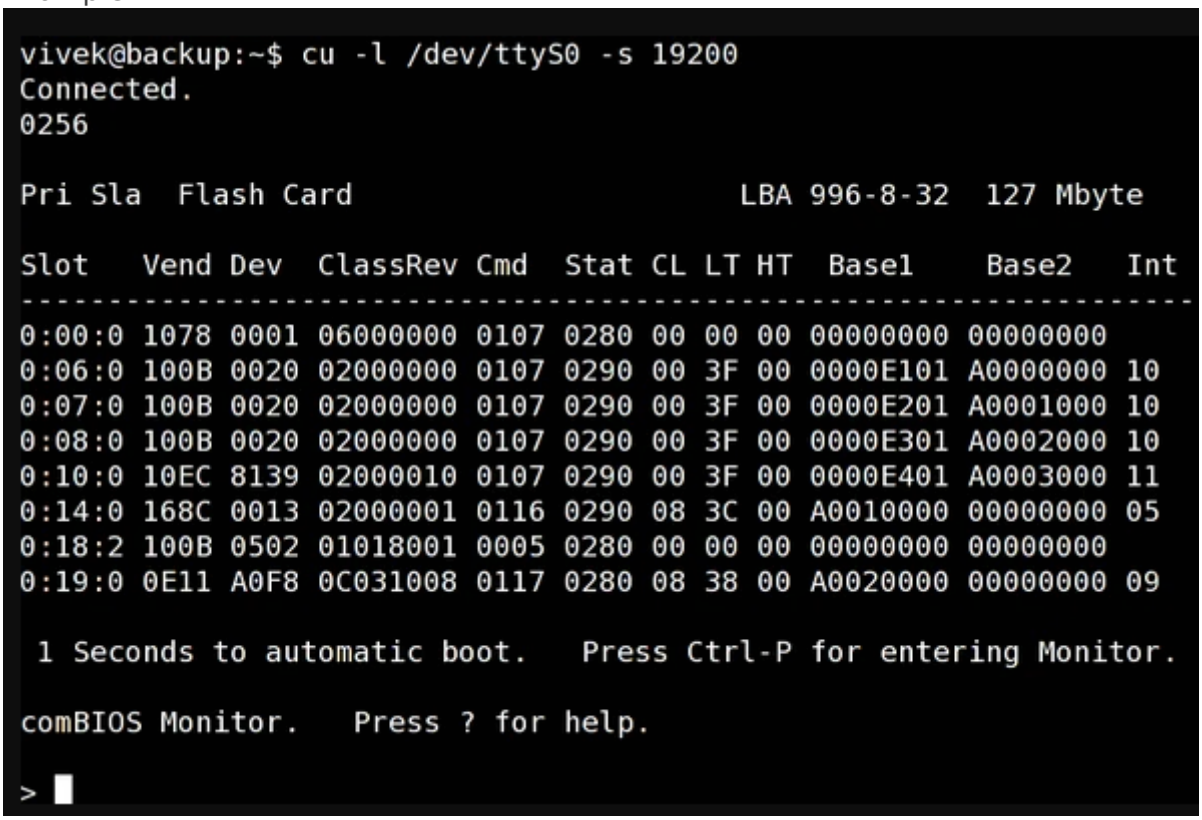
The boot loader has 2 main jobs:

- **To initialize the system to a basic level**
  - When the first act of the bootloader code are executed (**power-on** or **reset**), the system is in a very minimal state
  - DRAM controller in not set up so the main memory is not accessible
  - Storage interfaces (NAND flash controllers, MMC controllers,...) is **unavailable**
  - The **only** resources that are operational at the beginning are **a single CPU core**, a little **on-chip SRAM memory**, and the **boot ROM** (ROM-code)
- **To load the Kernel, establish the executation environment**: Bootloader will load Kernel image into RAM. Before move the control to Kernel, Bootloader do 2 things:
  - pass a pointer to a structure containing information about the **hardware configuration** (**device tree**, **Flattened Device Tree - FDT**...)
  - has to pass a pointer to the **kernel command line**

Once the kernel has begun executing, the bootloader is no longer needed and all the memory it was using can be reclaimed.

> **Subsidiary (maintainance mode)**: provide a simple **command line** interface through **Serial Console** for the maintainance (updating **boot configurations**, loading new **boot images** into memory or maybe **running diagnostics**)

Example:



1.2 The boot sequence

There are 3 phases:
Phase 1: ROM code

- Run immediately after a reset or power-on. It is loaded into the chip when it is manufactured, cannot be replaced by an open source equivalent
- DRAM is not initialize, ROM code is only use **Static Random Access Memory (SRAM)**, which does not require a memory controller (which varying in size from as little as 4KB to several hundred KB)
- ROM code search on storage device (NAND flash, (Serial Peripheral Interface) SPI flash, SD/eMMC card, UART, USB, Ethernet...) to read a small code, call **SPL (Secondary Program Loader)** , then load into **SRAM**

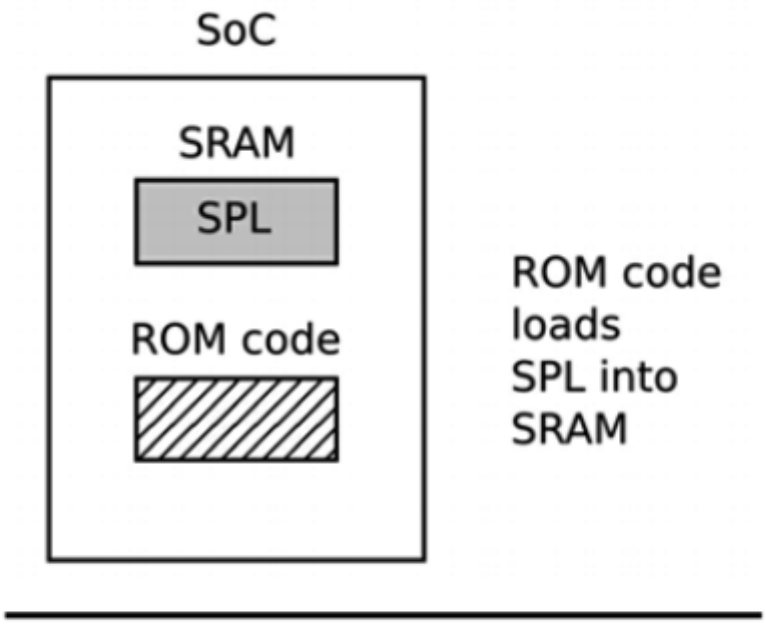
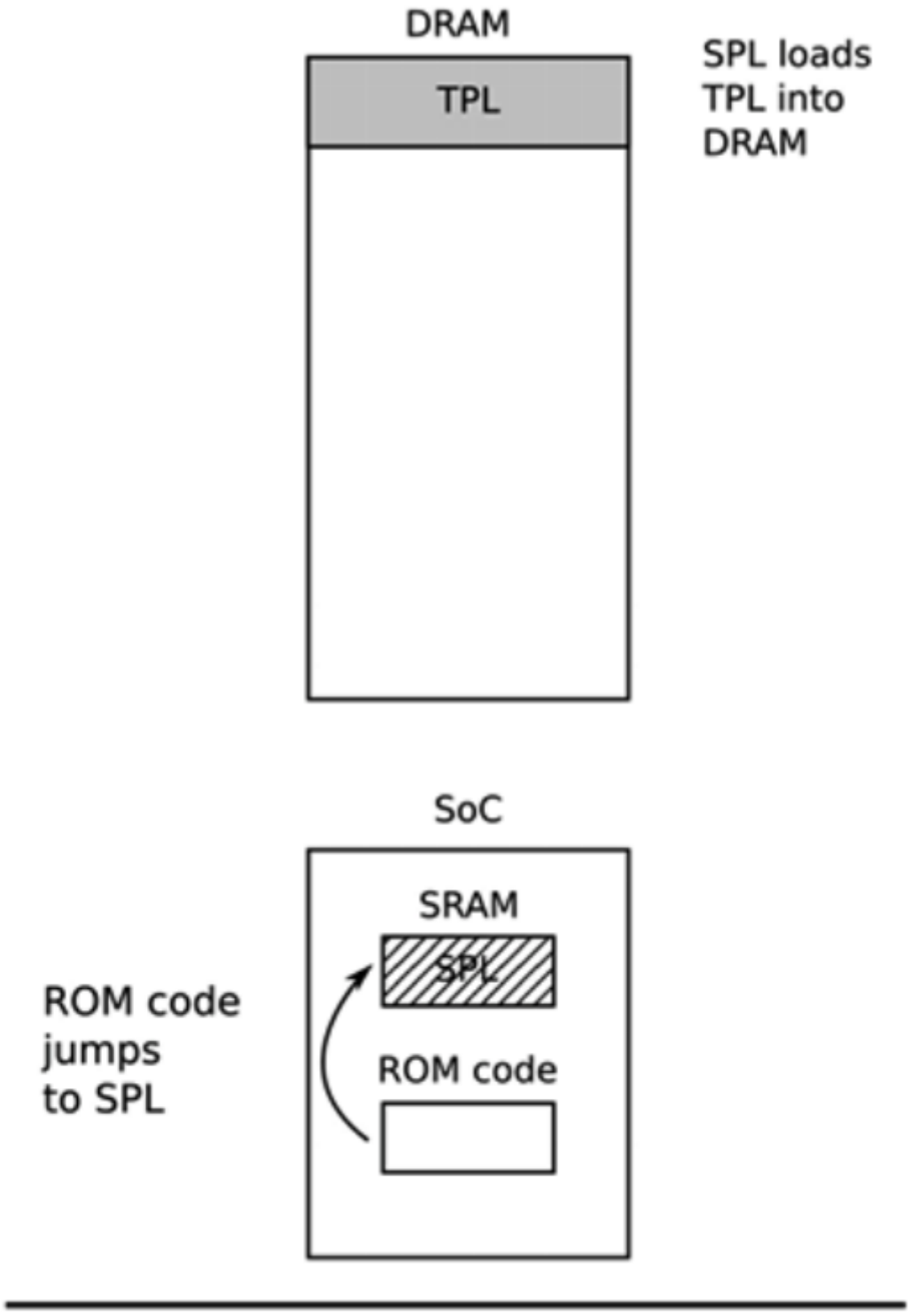


Figure 3.2 – Phase 1 – ROM code

Phase 2 – Secondary program loader (SPL)

- SPL run inside SRAM. The most important task of SPL is **configure DRAM controller** to active RAM
- Due to the limit of SPAM, SPL only has minimum function to load **TPL (Tertiary Program Loader)** - it is full Bootloader (ex: U-boot) - from storage device to DRAM



Phase 3 – Tertiary Program Loader (TPL) / Full Bootloader

- Run directly on main DRAM. At this point, we are running a full bootloader, such as U-Boot
- TPL provides the communication command line for read Kernel image (optional **Device Tree Blob file** and **initramfs**) , load it into RAM and change the controlling authority to Kernel

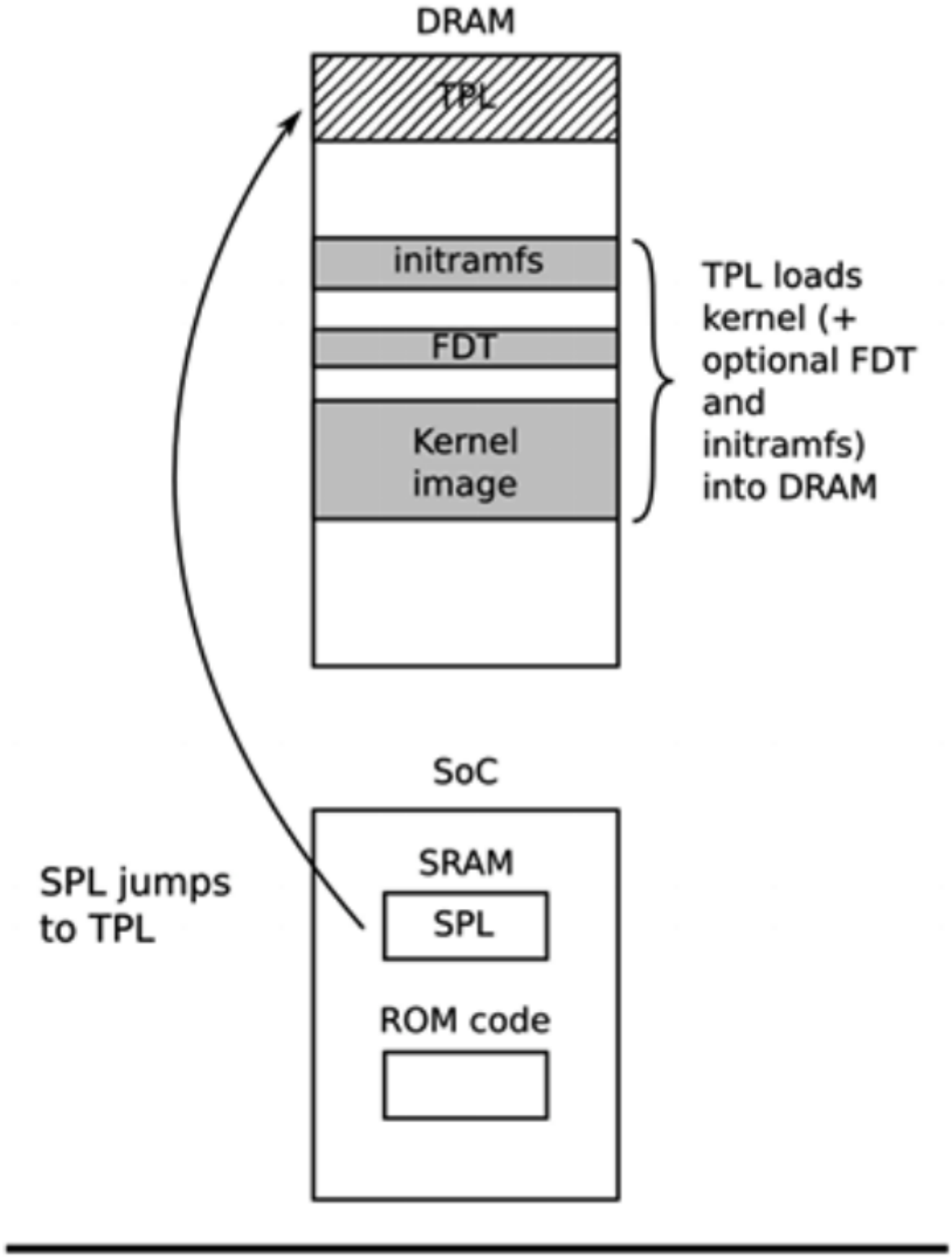


Figure 3.4 – Phase 3 – TPL

2. Introducing device trees

2.1 The definition and The source

Definition:

- A device tree is a flexible way of defining the hardware components of a computer system. Bear in mind that a device tree is just static data, not executable code.
- The device tree is loaded by the bootloader and passed to the kernel
- Although we can bundle (pack) the device tree with the kernel image itself for bootloaders, but this is not capable of loading them separately

Source: More use at Linux 3.8; replace for Embbed a **hard-coded platform data** into Kernel for allow one Kernel binary can run on a lot of hardware bases

2.2 Device tree basic

The device tree represents a computer system as a **tree**, including **nodes**
The device tree begins with a root node `/`, which contains subsequent nodes representing the hardware of the system (CPU, memory, bus, peripherals,...)

Simple Example:

```
/dts-v1/;
/{
    model = "TI AM335x BeagleBone";
    compatible = "ti,am33xx";
    #address-cells = <1>;
    #size-cells = <1>;
    cpus {                                          /* cpus node */
        #address-cells = <1>;
        #size-cells = <0>;
        cpu@0 {
            compatible = "arm,cortex-a8";
            device_type = "cpu";
            reg = <0>;
        };
    };
    memory@0x80000000 {                             /* memory modes */
        device_type = "memory";
        reg = <0x80000000 0x20000000>; /* 512 MB */
```

```
    };
};
```

- properties: Each node has the attributes type `name = value`(Ex: `compatible = "arm,cortex-a8";`). The name node has type `name@memory` (Ex: `cpu@0`, `memory@0x80000000`)
- important properties:
  - compatible: device strings. Type: `manufacturer, device_name` (Ex: `"ti,am33xx"`, `"arm,cortex-a8"`)
  - device_type: device layer (`cpu`, `memory`)

2.3 `reg` property

- Function: Represent the address range of register/physical memory and the size (length), which device occupies
- **Cell**: `reg` is written as zero or more 32-bit integers, call cell (Ex: `reg = <0x80000000 0x20000000>;` It mean `<begin, length> (address)`
- **Number of Cell**: `address-cells` or `size-cells` cannot represented in 32 bits. With on 64-bit addressing, you need 2 cells for each:

```
/{
    #address-cells = <2>;
    #size-cells = <2>;
    memory@80000000 {
        device_type = "memory";
        reg = <0x00000000 0x80000000 0 0x80000000>;
    };
};
```

`address-cells` and `size-cells` in **parent node** is specified the number of cell represent for address and size of **child node**

2.4 Labels and interrupts

- Labels `/`: for non-hierarchy connections, like to an interrupt controller, to a clock source, and to a voltage regulator. When compile, these labels are sometimes referred to an **unique numerical value** in a property called **phandle**
- Interrupt:
  - Interrupt the attribute declaration `interrupt-controller` and `interrupt-cells`

- interrupt the peripheral device, use `interrupt-parent = <&intc>` and `interrupts = <interrupt_number>`

2.5 Device tree include files

- Re-use source code (`.dtsi`)
- We can use `/include/ "file.dtsi"` or for C language `#include "file.dtsi"`. Whenever use Kbuild, source file will pass through C preprocessor, allow listen the `#define` constant between Kernel resource and Device Tree
- Overlays: When include files, nodes in outside layer can **overlay** on inside nodes for overwrite or change the attribute (Ex: change `status = "disabled"` in SoC file to `status = "okay"` in specified file in board)

2.6 Compiling a device tree

- Tool: We use Device Tree Compiler - `dtc` for convert from text file `.dts` to binary file `.dtb`
- Execute: Bootloader and Kernel only read directly on binary file `.dtb`

3. U-boot

Revision #2
Created 2026-10-22 02:42:44 UTC by Lê Hoài Nam
Updated 2026-10-22 10:28:20 UTC by Lê Hoài Nam